isolation, but a guarantee from the virtual machine runtime is another level of confidence.

Process Memory Architecture

Each process in Elixir has its own memory space. This consists of a heap and a stack that grow toward each other. Eventually, if they are unable to grow, the BEAM will allocate more memory to the process.

Processes start off with a fairly small amount of memory. On my computer, I see 2608 bytes taken up for a brand new process:

```
$ iex -S mix
iex> pid = Examples.Spawn.Infinite.start()
iex> Process.info(pid, :memory)
{:memory, 2608}
```

Process.info(pid) is a useful source of information about any process that's actively running. It tells you things like heap size, stack size, reductions (which roughly equate to CPU usage), and the number of unprocessed messages. Here, we used Process.info/2 to return a focused version of the available data.

Data in Elixir is copied between processes. So, if you send a message to a process, that memory will be duplicated and then passed as a message. This has benefits for small bits of data, but it would be a bit of a waste to copy every single message between processes. Elixir has a little trick up its sleeve to optimize copies.

Elixir uses a binary heap[3] to globally store large (> 64 bytes) binary data. This binary heap is shared between processes and uses reference counting to determine when the memory can be cleaned up. Because the BEAM uses immutable data, you don't need to worry about this causing bugs in your application. The memory here is safe to use and can be referenced by multiple processes without fear.

The small memory size of processes is part of what makes them easy to spawn and destroy. But sometimes you'll find yourself debugging a problem where too much memory is being used. So, let's cover how garbage collection works.

Garbage Collection

Garbage collection isn't fun, right? Actually, the BEAM's garbage collector is rather interesting. I wrote about this in fairly deep detail in *Real-Time Phoenix [Bus20]*, so we'll cover much less in this book.

3. https://www.erlang.org/doc/apps/erts/garbagecollection#binary-heap